

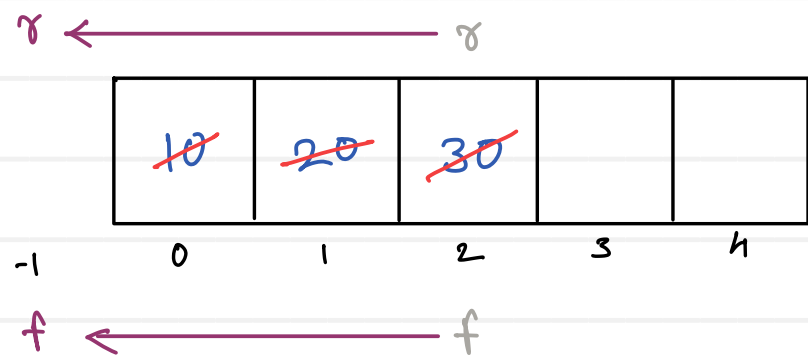
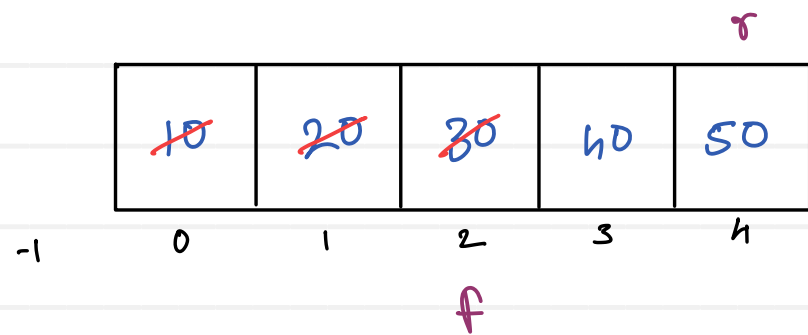


Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

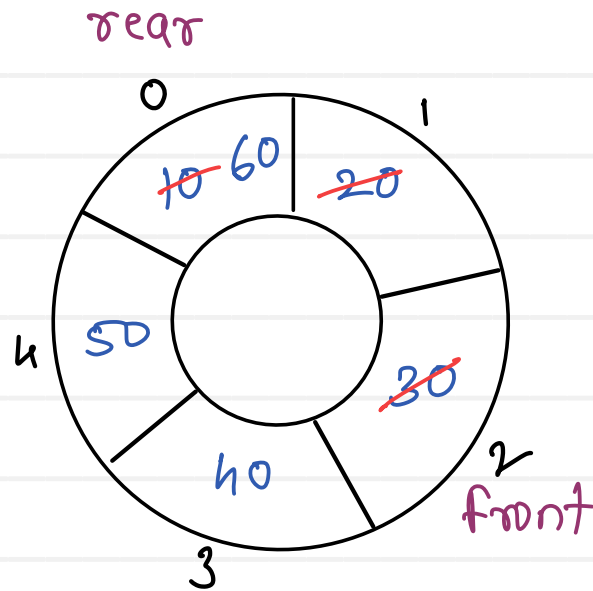


- if rear is at last index of array & initial few locations of array are empty, we can not reuse them untill queue is totally empty. This leads to poor memory utilization.

```
int pop() {
    int val = arr[front+1];
    front++;
    if (front == rear) // queue is empty
        front = rear = -1;
    return val;
}
```

Circular queue

Capacity = 5
front = rear = -1



Operations :

1) insert/add/enqueue/push :

- reposition rear (inc)
- add value at rear index

2) remove/delete/dequeue/pop :

- reposition front (inc)

3) peek :

- read data from front end (front+1)

$$\text{front} = (\text{front} + 1) \% \text{size}$$

$$\text{rear} = (\text{rear} + 1) \% \text{size}$$

$$\text{front} = \text{rear} = -1$$

$$= (-1 + 1) \% 5 = 0$$

$$= (0 + 1) \% 5 = 1$$

$$= (1 + 1) \% 5 = 2$$

$$= (2 + 1) \% 5 = 3$$

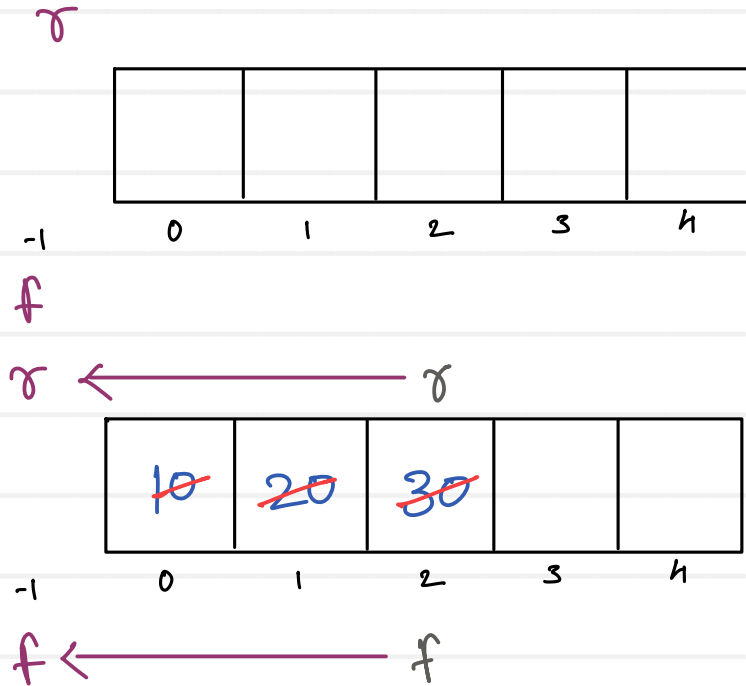
$$= (3 + 1) \% 5 = 4$$

$$= (4 + 1) \% 5 = 0$$

All operations in circular queue are performed in $O(1)$ time complexity.

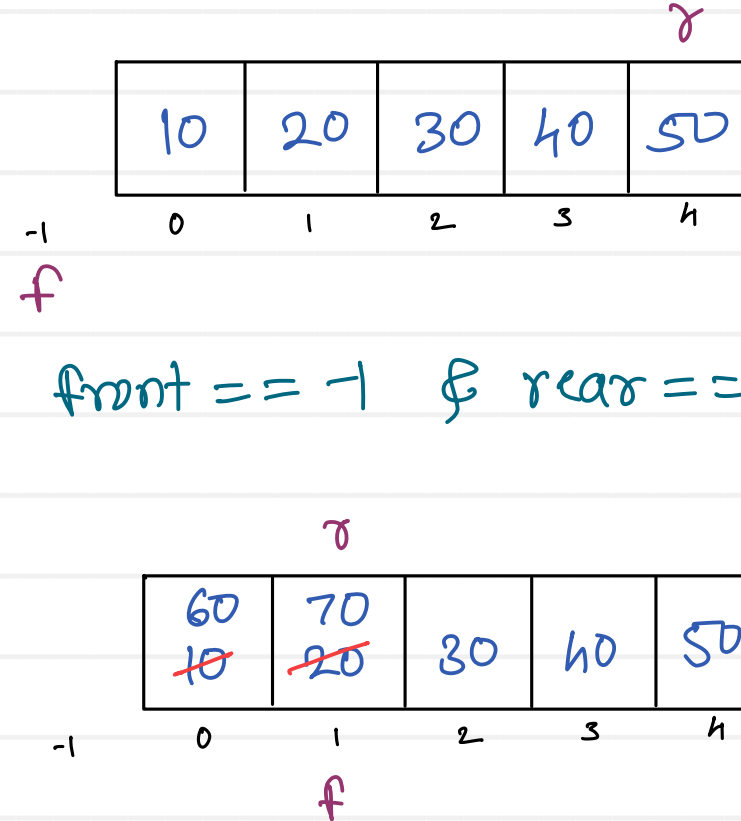
Circular queue - Conditions

Empty



$front == rear \ \&\& \ rear == -1$

Full



$front == rear \ \&\& \ rear != -1$

Circular queue using count method

CircularQueue :

int arr [];

int size, count;

int front, rear;

public CircularQueue(size) {

 this.size = size;

 arr = new int[size];

 front = rear = -1;

 count = 0;

}

public void offer(int val) {

 =

 count++;

}

public int poll() {

 =

 count--;

}

public boolean isEmpty() {

 return count == 0;

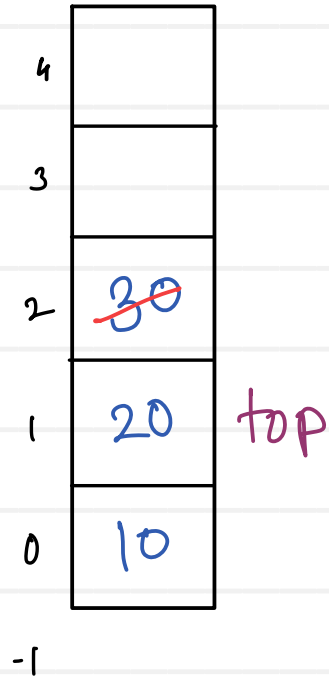
}

public boolean isFull() {

 return count == size;

}

- Stack is a linear data structure which has only one end - top
- Data is inserted and removed from top end only.
- Stack works on principle of "Last In First Out" / "LIFO"



- top always points to last inserted value/data.

Operations:

1. insert / push:

- reposition top (++)
- add value at top index

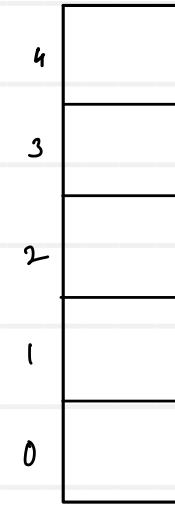
2. remove / pop:

- reposition top (--)

3. peek:

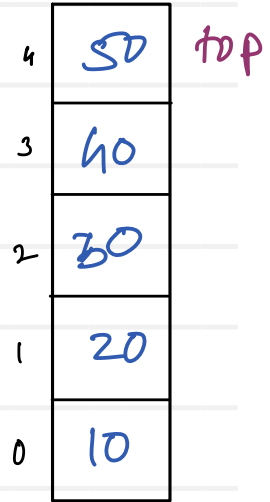
- read value from top index

Empty



top = -1

Full



top = size - 1

All operations of stack will be performed in $O(1)$ time complexity.

Stack

- Parenthesis balancing
- Expression conversion and evaluation
- Function calls
- Used in advanced data structures for traversing
- **Expression conversion and evaluation:**
 - Infix to postfix
 - Infix to prefix
 - Postfix evaluation
 - Prefix evaluation

Queue

- Jobs submitted to printer
- In Network setups – file access of file server machine is given to First come First serve basis
- Calls are placed on a queue when all operators are busy
- Used in advanced data structures to give efficiency.
- Process waiting queues in OS

Expression :

combination of operands & operators

Types of expression :

1. Infix : $a + b$ ← human

2. Prefix : $+ a b$

3. Postfix : $a b +$

} ← computer → CPU → ALU (hardware is designed in such a way that, it understands either prefix or postfix expression).

Operator priorities :

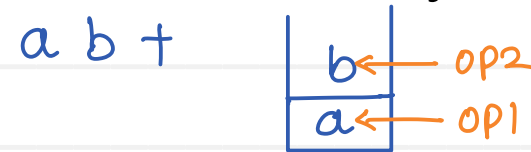
()
\$ (power)
* / %
+ -



1. Infix to postfix } used by compiler
2. Infix to prefix }
3. postfix evaluation } used by ALU
4. prefix evaluation }

Postfix Evaluation

- Process each element of postfix expression from left to right
- If element is operand
 - Push it on a stack
- If element is operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op2 – first popped element
 - Op1 – second popped element
 - Perform current element (Operator) operation between Op1 and Op2
 - Again push back result onto the stack
- When single value will remain on stack, it is final result
- e.g. 4 5 6 * 3 / + 9 + 7 -



Postfix evaluation

Postfix expression : 5 9 + 4 8 6 2 / - * - 1 7 3 - \$ +

l $\xrightarrow{\hspace{10em}}$ r

result = -5

$$-6 + 1 = -5$$

$$1 \$ 4 = 1$$

$$7 - 3 = 4$$

$$14 - 20 = -6$$

$$4 * 5 = 20$$

$$8 - 3 = 5$$

$$6 / 2 = 3$$

$$5 + 9 = 14$$

-5 $\leftarrow$ result

1
4
8
7
1
-6
20
5
3
2
6
8
4
14
9
5

stack

'7' $\rightarrow$ ASCII value

$$0 \rightarrow 9 = 48 \rightarrow 57$$

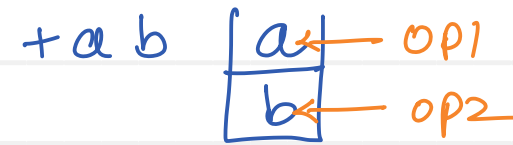
$$'0' - '0' = 48 - 48 = 0$$

$$'1' - '0' = 49 - 48 = 1$$

⋮

Prefix Evaluation

- Process each element of prefix expression from right to left
- If element is operand
 - Push it on a stack
- If element is operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op1 – first popped element
 - Op2 – second popped element
 - Perform current element (Operator) operation between Op1 and Op2
 - Again push back result onto the stack
- When single value will remain on stack, it is final result
- e.g. - + + 4 / * 5 6 3 9 7



Prefix evaluation

Prefix expression : + - + 5 9 * 4 - 8 / 6 2 \$ 1 - 7 3

l ← ————— r

result = -5

$$-6 + 1 = -5$$

$$14 - 20 = -6$$

$$5 + 9 = 14$$

$$4 * 5 = 20$$

$$8 - 3 = 5$$

$$6 / 2 = 3$$

$$1 \$ 4 = 1$$

$$7 - 3 = 4$$

-5	← result
-6	
14	
5	
9	
20	
4	
5	
8	
3	
6	
2	
1	
1	
4	
7	
3	

stack

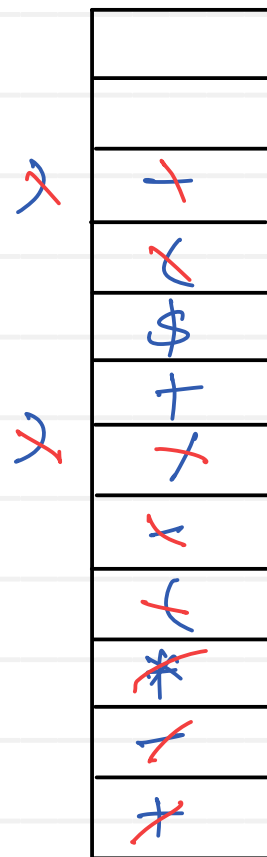
- Process each element of infix expression from left to right
- If element is Operand
 - Append it to the postfix expression
- If element is Operator
 - If priority of topmost element (Operator) of stack is greater or equal to current element (Operator), pop topmost element from stack and append it to postfix expression
 - Repeat above step if required
 - Push element on stack
- Pop all remaining elements (Operators) from stack one by one and append them into the postfix expression
- e.g. $a * b / c * d + e - f * h + i$

Infix to Postfix conversion

Infix expression : $5 + 9 - 4 * (8 - 6 / 2) + 1 \$ (7 - 3)$

$\xrightarrow{\quad}$

Postfix expression : $59+4862/-* -173-\$+$



stack

- for opening '('
push it on stack

- for closing ')'
pop operators from
stack one by one &
append into postfix
expression until
opening comes.

discard both '(' & ')'

Infix to Prefix Conversion

- Process each element of infix expression from right to left
- If element is Operand
 - Append it to the prefix expression
- If element is Operator
 - If priority of topmost element of stack is greater than current element (Operator), pop topmost element from stack and append it to prefix expression
 - Repeat above step if required
 - Push element on stack
- Pop all remaining elements (Operators) from stack one by one and append them into the prefix expression
- Reverse prefix expression
- e.g. $a * b / c * d + e - f * h + i$

Infix to Prefix conversion

Infix expression : $5 + 9 - 4 * (8 - 6 / 2) + 1 \$ (7 - 3)$

$\leftarrow$ $\rightarrow$

Expression : $37-1\$26/8-4*95+-+$

Prefix expression : $+ - + 5 9 * 4 - 8 / 6 2 \$ 1 - 7 3$

Infix : "10 + 20"

prefix : "+ 10 20" : prefix.split(" ")

postfix : "10 20 +"

$\downarrow$
 { "+", "10", "20" }

+
-
*
/
x
)
+
\$
-
)

stack

- for closing ')' push it on stack

- for opening '(' pop operators from stack one by one & append into postfix expression until closing comes.

discard both '(' & ')'

Prefix to Postfix

- Process each element of prefix expression from right to left
- If element is an Operand
 - Push it on to the stack
- If element is an Operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op1 – first popped element
 - Op2 – second popped element
 - Form a string by concatenating Op1, Op2 and Opr (element)
 - String = “Op1+Op2+Opr”, push back on to the stack
- Repeat above two steps until end of prefix expression.
- Last remaining on the stack is postfix expression
- e.g. $* + a b - c d$

Postfix to Infix

- Process each element of postfix expression from left to right
- If element is an Operand
 - Push it on to the stack
- If element is an Operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op2 – first popped element
 - Op1 – second popped element
 - Form a string by concatenating Op1, Opr (element) and Op2
 - String = “Op1+Opr+Op2”, push back on to the stack
- Repeat above two steps until end of postfix expression.
- Last remaining on the stack is infix expression
- E.g. a b c - + d e - f g - h + / *